

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import r2_score, mean_absolute_error
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LinearRegression
from sklearn.impute import SimpleImputer

df = pd.read_csv('/content/train.csv')

y = df['Sales']
X = df.drop(columns=['Sales'])

cat_cols = X.select_dtypes(include=['object']).columns.tolist()
num_cols = X.select_dtypes(include=['int64', 'float64']).columns.tolist()

pre = ColumnTransformer([
    ('cat', Pipeline([
        ('impute', SimpleImputer(strategy='most_frequent')),
        ('onehot', OneHotEncoder(max_categories=50, handle_unknown='ignore'))
    ]), cat_cols),
    ('num', Pipeline([
        ('impute', SimpleImputer(strategy='median')),
        ('scale', StandardScaler())
    ]), num_cols)
])

models = {
    'LinearRegression': LinearRegression(),
    'DecisionTree': DecisionTreeRegressor(),
    'RandomForest': RandomForestRegressor(n_estimators=60)
}

results = {}

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

for name, model in models.items():
    pipe = Pipeline([('pre', pre), ('model', model)])
    pipe.fit(X_train, y_train)
    preds = pipe.predict(X_test)
    r2 = r2_score(y_test, preds)
    mae = mean_absolute_error(y_test, preds)
    results[name] = {'R2': r2, 'MAE': mae}

results

```

```

{'LinearRegression': {'R2': 0.1700616274640867, 'MAE': 252.8676590389218},
 'DecisionTree': {'R2': -0.1862477970030909, 'MAE': 286.9958784183674},
 'RandomForest': {'R2': 0.06443674848474212, 'MAE': 244.5621242117347}}

```

```

# =====
# COMPLETE MACHINE LEARNING MODEL COMPARISON SCRIPT
# Prints results like the screenshot you shared
# =====

import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.impute import SimpleImputer

# Models
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor

```

```

from sklearn.neighbors import KNeighborsRegressor
from sklearn.svm import SVR

# XGBoost
!pip install xgboost -q
from xgboost import XGBRegressor

# Neural network
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# =====
# Load dataset
# =====
df = pd.read_csv("/content/train.csv") # <-- change if needed

y = df["Sales"]
X = df.drop(columns=["Sales"])

# Identify feature types
cat_cols = X.select_dtypes(include=['object']).columns.tolist()
num_cols = X.select_dtypes(include=['int64', 'float64']).columns.tolist()

# Preprocessing
preprocess = ColumnTransformer([
    ('cat', Pipeline([
        ('impute', SimpleImputer(strategy='most_frequent')),
        ('onehot', OneHotEncoder(handle_unknown='ignore'))
    ]), cat_cols),

    ('num', Pipeline([
        ('impute', SimpleImputer(strategy='median')),
        ('scale', StandardScaler())
    ]), num_cols)
])

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# =====
# Helper function for printing model performance
# =====
def evaluate_model(name, model):
    pipe = Pipeline([('pre', preprocess), ('model', model)])
    pipe.fit(X_train, y_train)
    preds = pipe.predict(X_test)

    mse = mean_squared_error(y_test, preds)
    r2 = r2_score(y_test, preds)

    print(f"\n{name} Performance:")
    print(f"  Mean Squared Error: {mse:.4f}")
    print(f"  R-squared: {r2:.4f}")

# =====
# Train and evaluate classical ML models
# =====
evaluate_model("Linear Regression", LinearRegression())
evaluate_model("Decision Tree", DecisionTreeRegressor(max_depth=10))
evaluate_model("Random Forest", RandomForestRegressor(n_estimators=150))
evaluate_model("Gradient Boosting", GradientBoostingRegressor())
evaluate_model("KNN", KNeighborsRegressor(n_neighbors=5))
evaluate_model("SVM", SVR())

# =====
# XGBoost
# =====
evaluate_model("XGBoost", XGBRegressor(
    n_estimators=200,
    learning_rate=0.1,
    max_depth=6,
    subsample=0.7,
    colsample_bytree=0.7
))

```



```
# =====
# Neural Network (Keras)
# =====

# Prepare data for NN (manual transform)
X_processed = preprocess.fit_transform(X)
X_train_nn, X_test_nn, y_train_nn, y_test_nn = train_test_split(
    X_processed, y, test_size=0.2, random_state=42
)

model_nn = Sequential([
    Dense(64, activation='relu', input_dim=X_processed.shape[1]),
    Dense(32, activation='relu'),
    Dense(1)
])

model_nn.compile(optimizer='adam', loss='mse')

print("\nTraining Neural Network...")
model_nn.fit(X_train_nn, y_train_nn, epochs=20, batch_size=32, verbose=0)

nn_preds = model_nn.predict(X_test_nn)
nn_mse = mean_squared_error(y_test_nn, nn_preds)
nn_r2 = r2_score(y_test_nn, nn_preds)

print("\nNeural Network Performance:")
print(f" Mean Squared Error: {nn_mse:.4f}")
print(f" R-squared: {nn_r2:.4f}")
```

Linear Regression Performance:
 Mean Squared Error: 622900.8447
 R-squared: 0.0681

Decision Tree Performance:
 Mean Squared Error: 558060.6101
 R-squared: 0.1651

Random Forest Performance:
 Mean Squared Error: 476599.2705
 R-squared: 0.2870

Gradient Boosting Performance:
 Mean Squared Error: 509715.9673
 R-squared: 0.2374

KNN Performance:
 Mean Squared Error: 650824.8022
 R-squared: 0.0263

SVM Performance:
 Mean Squared Error: 703892.9395
 R-squared: -0.0531

XGBoost Performance:
 Mean Squared Error: 465431.0529
 R-squared: 0.3037

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input\_shape` to `Dense` layer. It is not supported.
 super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Training Neural Network...
 62/62 ————— 0s 6ms/step

Neural Network Performance:
 Mean Squared Error: 496267.9464
 R-squared: 0.2575



